CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology (CSPIT) Department of Computer Science & Engineering

Date: 23/10/2026

Laboratory Manual

Academic Year : 2026-27	**Semester** : 3rd	**Course code** : ITUE203	**Course name** : WEB DEVELOPMENT FRAMEWORKS

Practical – 14

PROBLEM STATEMENT :

Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role. Implement an immutable audit log trail recording all mutating actions (CREATE, UPDATE, DELETE, ROLE_SWITCH) with user role, action, details, client IP address, and timestamps.

Objective :

- Develop an administrative monitoring dashboard featuring key metrics: total users, student count, event count, and latest registrations
- Enforce strict Role-Based Access Control (RBAC) restricting administrative views to authorized roles
- Dynamically render navigation menus matching user privilege levels (user, editor, admin)
- Record critical system and administrative actions in an audit_logs table (timestamp, action, user/role, details, IP address)
- Integrate asynchronous AJAX/Fetch updates for inventory items, user status toggling, and audit log tracking without page reloads

Code :

A. schema.sql (Database DDL with Users, Students, Events, Registrations & Audit Logs)

```
-- Practical 14: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log
-- Database Schema: dashboard_db

CREATE DATABASE IF NOT EXISTS dashboard_db;
USE dashboard_db;

-- 1. Dynamic Navigation Menu Table (RBAC restricted)
CREATE TABLE IF NOT EXISTS menu_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(50) NOT NULL,
    link VARCHAR(100) NOT NULL,
    min_role ENUM('user', 'editor', 'admin') DEFAULT 'user'
);

-- 2. System Users Table with Roles and Status
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    role ENUM('user', 'editor', 'admin') NOT NULL DEFAULT 'user',
```

A. schema.sql (Seed Records for Audit Logs, Users & Registrations)

```
CURRENT_TIMESTAMP
);

-- 6. Items Table for Inventory CRUD
CREATE TABLE IF NOT EXISTS items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- 7. Audit Logs Table for Administrative Traceability
CREATE TABLE IF NOT EXISTS audit_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_role VARCHAR(20) NOT NULL DEFAULT 'admin',
    action VARCHAR(50) NOT NULL,
    details TEXT NOT NULL,
    ip_address VARCHAR(45) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Seed Data: Menu Items
INSERT INTO menu_items (title, link, min_role) VALUES
('Dashboard', '#dashboard', 'user'),
('View Inventory', '#inventory', 'user'),
('Manage Content', '#content', 'admin'),
('Audit Logs', '#audit', 'admin'),
('User Management', '#users', 'admin'),
('System Settings', '#settings', 'admin');

-- Seed Data: Users
INSERT INTO users (username, email, role, status) VALUES
('admin_bhavy', 'bhavy.admin@charusat.edu', 'admin', 'Active'),
('editor_patel', 'editor.patel@charusat.edu', 'editor', 'Active'),
('student_vansh', 'vansh.student@charusat.edu', 'user', 'Active'),
('guest_user', 'guest@charusat.edu', 'user', 'Inactive');

-- Seed Data: Students
INSERT INTO students (name, email, department) VALUES
('Bhavy Gol', '25cs016@charusat.edu', 'CSE'),
('Aarav Mehta', 'aarav@charusat.edu', 'CSE'),
('Diya Shah', 'diya@charusat.edu', 'CSE'),
('Rohan Joshi', 'rohan@charusat.edu', 'ECE'),
('Ananya Dave', 'ananya@charusat.edu', 'AI & ML');

-- Seed Data: Events
INSERT INTO events (title, event_date, location, status) VALUES
('Tech Fest 2026', '2026-11-20', 'Main Auditorium', 'Open'),
('Web Dev Hackathon', '2026-12-05', 'CSE Lab 3', 'Open'),
('AI Conference', '2026-12-15', 'Seminar Hall A', 'Closed');

-- Seed Data: Registrations
INSERT INTO registrations (student_name, event_title) VALUES
('Bhavy Gol', 'Web Dev Hackathon'),
('Aarav Mehta', 'Tech Fest 2026'),
('Diya Shah', 'Web Dev Hackathon'),
('Ananya Dave', 'AI Conference');

-- Seed Data: Items
INSERT INTO items (name, category, price) VALUES
```

B. db.php (Database Connection Script with SQLite Fallback)

```
<?php
// Practical 14: PDO Database Connection with JSON Error Handling & SQLite Fallback
$host = 'localhost';
$db   = 'dashboard_db';
$user = 'root';
$pass = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8mb4", $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (PDOException $e) {
    // Fallback to SQLite if MySQL is not available
    $sqlitePath = __DIR__ . '/dashboard.sqlite';
    $isNew = !file_exists($sqlitePath);
    $pdo = new PDO("sqlite:$sqlitePath", null, null, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
    if ($isNew) {
        $pdo->exec("CREATE TABLE IF NOT EXISTS menu_items (id INTEGER PRIMARY KEY AUTOINCREMEN
T, title TEXT NOT NULL, link TEXT NOT NULL, min_role TEXT DEFAULT 'user');");
        $pdo->exec("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, us
ername TEXT NOT NULL, email TEXT NOT NULL, role TEXT DEFAULT 'user', status TEXT DEFAULT 'Active',
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP);");
        $pdo->exec("CREATE TABLE IF NOT EXISTS students (id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL, email TEXT NOT NULL, department TEXT NOT NULL, created_at DATETIME DEFAULT CUR
RENT_TIMESTAMP);");
        $pdo->exec("CREATE TABLE IF NOT EXISTS events (id INTEGER PRIMARY KEY AUTOINCREMENT, t
itle TEXT NOT NULL, event_date TEXT NOT NULL, location TEXT NOT NULL, status TEXT DEFAULT 'Ope
n');");
        $pdo->exec("CREATE TABLE IF NOT EXISTS registrations (id INTEGER PRIMARY KEY AUTOINCRE
MENT, student_name TEXT NOT NULL, event_title TEXT NOT NULL, registered_at DATETIME DEFAULT CURREN
T_TIMESTAMP);");
        $pdo->exec("CREATE TABLE IF NOT EXISTS items (id INTEGER PRIMARY KEY AUTOINCREMENT, na
```

C. api.php (RESTful API - RBAC Menu, Stats Aggregation & Audit Logging)

```
<?php
// Practical 14: Admin Dashboard RESTful JSON API with RBAC, Aggregates & Audit Logging
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST");
header("Access-Control-Allow-Headers: Content-Type");

require_once 'db.php';

$action = $_GET['action'] ?? '';
$input  = json_decode(file_get_contents('php://input'), true) ?? [];

// Helper function to record audit log entries
function record_audit($pdo, $role, $action, $details) {
    try {
        $ip = $_SERVER['REMOTE_ADDR'] ?? '127.0.0.1';
        $stmt = $pdo->prepare("INSERT INTO audit_logs (user_role, action, details, ip_address) VAL
UES (?, ?, ?, ?)");
        $stmt->execute([$role, $action, $details, $ip]);
    } catch (Exception $e) {
        // Silently continue if audit log insert fails
    }
}

switch ($action) {
    // ------------------------------------------------------------
    // 1. DYNAMIC NAVIGATION MENU (RBAC)
    // ------------------------------------------------------------
    case 'get_menu':
        $role = $_GET['role'] ?? 'user';
        $rolesAllowed = ['user'];
        if ($role === 'editor') {
            $rolesAllowed = ['user', 'editor'];
        } elseif ($role === 'admin') {
            $rolesAllowed = ['user', 'editor', 'admin'];
        }
        $placeholders = implode(',', array_fill(0, count($rolesAllowed), '?'));
        $stmt = $pdo->prepare("SELECT title, link FROM menu_items WHERE min_role IN ($placeholder
s)");
        $stmt->execute($rolesAllowed);
        echo json_encode(['status' => 'success', 'data' => $stmt->fetchAll()]);
        break;

    // ------------------------------------------------------------
    // 2. DASHBOARD METRICS / AGGREGATE COUNTS
    // ------------------------------------------------------------
    case 'stats':
        $userCount = $pdo->query("SELECT COUNT(*) AS total FROM users")->fetch()['total'] ?? 0;
        $studentCount = $pdo->query("SELECT COUNT(*) AS total FROM students")->fetch()['total'] ??
0;
        $eventCount = $pdo->query("SELECT COUNT(*) AS total FROM events")->fetch()['total'] ?? 0;
        $regCount = $pdo->query("SELECT COUNT(*) AS total FROM registrations")->fetch()['total']
?? 0;

        echo json_encode([
            'status' => 'success',
            'data' => [
                'users'         => (int)$userCount
```

C. api.php (RESTful API - CRUD Operations & State Toggling with Automatic Audit Logging)

```
,
                'students'      => (int)$studentCount,
                'events'        => (int)$eventCount,
                'registrations' => (int)$regCount,
            ]
        ]);
        break;

    // ------------------------------------------------------------
    // 3. AUDIT LOGS RETRIEVAL
    // ------------------------------------------------------------
    case 'get_audit_logs':
        $stmt = $pdo->query("SELECT * FROM audit_logs ORDER BY id DESC LIMIT 15");
        echo json_encode(['status' => 'success', 'data' => $stmt->fetchAll()]);
        break;

    // ------------------------------------------------------------
    // 4. CRUD: READ ITEMS
    // ------------------------------------------------------------
    case 'read':
        $stmt = $pdo->query("SELECT * FROM items ORDER BY id DESC");
        echo json_encode(['status' => 'success', 'data' => $stmt->fetchAll()]);
        break;

    // ------------------------------------------------------------
    // 5. CRUD: CREATE ITEM (WITH AUDIT LOG)
    // ------------------------------------------------------------
    case 'create':
        if (!empty($input['name']) && !empty($input['category']) && isset($input['price'])) {
            $stmt = $pdo->prepare("INSERT INTO items (name, category, price) VALUES (?, ?, ?)");
            $stmt->execute([$input['name'], $input['category'], $input['price']]);
            $newId = $pdo->lastInsertId();
            record_audit($pdo, $input['role'] ?? 'admin', 'CREATE_ITEM', "Created item: {$input['n
ame']} ($newId) in {$input['category']}");
            echo json_encode(['status' => 'success', 'message' => 'Item created successfully']);
        } else {
            echo json_encode(['status' => 'error', 'message' => 'Missing required fields']);
        }
        break;

    // ------------------------------------------------------------
    // 6. CRUD: UPDATE ITEM (WITH AUDIT LOG)
    // ------------------------------------------------------------
    case 'update':
        if (!empty($input['id']) && !empty($input['name']) && !empty($input['category']) && isset
($input['price'])) {
            $stmt = $pdo->prepare("UPDATE items SET name = ?, category = ?, price = ? WHERE id =
?");
            $stmt->execute([$input['name'], $input['category'], $input['price'], $input['id']]);
            record_audit($pdo, $input['role'] ?? 'admin', 'UPDATE_ITEM', "Updated item #{$input['i
d']}: {$input['name']} to \${$input['price']}");
            echo json_encode(['status' => 'success', 'message' => 'Item updated successfully']);
        } else {
            echo json_encode(['status' => 'error', 'message' => 'Invalid or missing data']);
        }
        break;

    // ------------------------------------------------------------
    // 7. CRUD: DELETE ITEM (WITH AUDIT LOG)
    // ------------------------------------------------------------
```

D. index.html (Admin Monitoring Dashboard UI Layout & Audit Log Viewer)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Dashboard with RBAC & Audit Log</title>
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-ser
if; margin: 20px; background: #f4f6f9; color: #333; }
        header { display: flex; justify-content: space-between; align-items: center; background: #
212529; padding: 14px 24px; color: #fff; border-radius: 6px; box-shadow: 0 2px 5px rgba(0,0,0,0.1
5); }
        nav ul { list-style: none; margin: 0; padding: 0; display: flex; gap: 18px; }
        nav a { color: #f8f9fa; text-decoration: none; font-weight: 600; font-size: 14px; transiti
on: color 0.2s; }
        nav a:hover { color: #0d6efd; }
        .role-box { display: flex; align-items: center; gap: 8px; font-size: 14px; }
        .role-box select { padding: 6px 12px; border-radius: 4px; border: 1px solid #ced4da; backg
round: #fff; font-weight: 500; }

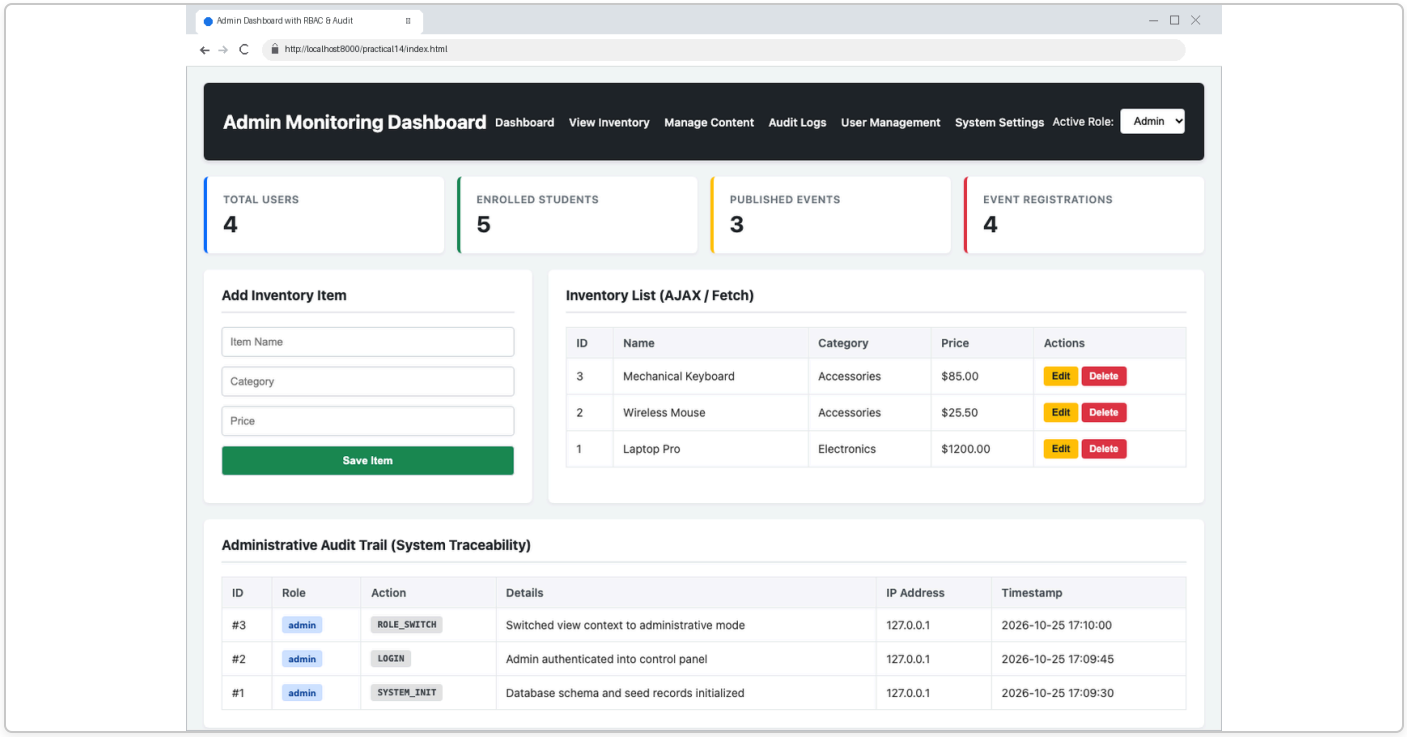
        /* Metrics Grid */
        .metrics-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1f
r)); gap: 16px; margin-top: 20px; }
        .metric-card { background: #fff; padding: 20px; border-radius: 6px; box-shadow: 0 2px 4px
rgba(0,0,0,0.06); border-left: 4px solid #0d6efd; display: flex; flex-direction: column; }
        .metric-card.users { border-left-color: #0d6efd; }
        .metric-card.students { border-left-color: #198754; }
        .metric-card.events { border-left-color: #ffc107; }
        .metric-card.regs { border-left-color: #dc3545; }
        .metric-title { font-size: 13px; color: #6c757d; font-weight: 600; text-transform: upperca
se; letter-spacing: 0.5px; }
        .metric-value { font-size: 28px; font-weight: 700; margin-top: 6px; color: #212529; }

        /* Main Layout */
        .container { margin-top: 20px; display: grid; grid-template-columns: 1fr 2fr; gap: 20px; }
        .full-width { margin-top: 20px; }
        .card { background: #fff; padding: 22px; border-radius: 6px; box-shadow: 0 2px 4px rgba(0,
0,0,0.06); }
        .card h3 { margin-top: 0; font-size: 17px; border-bottom: 2px solid #e9ecef; padding-botto
m: 10px; color: #212529; }

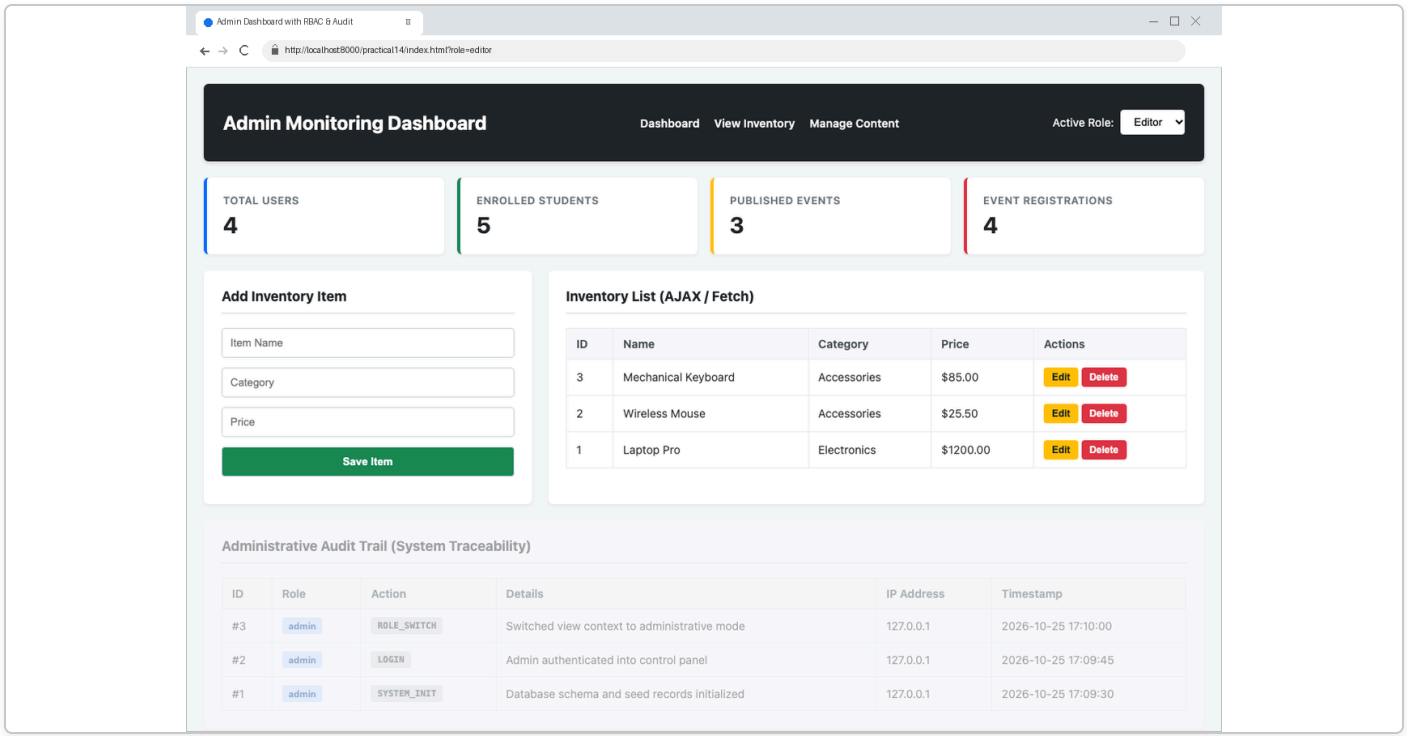
        /* Forms */
        form input, form select, form button { width: 100%; padding: 10px; margin-bottom: 12px; bo
x-sizing: border-box; border: 1px solid #ced4da; border-radius: 4px; }
        form input:focus { outline: none; border-col
```

OUTPUT & SCREENSHOTS:

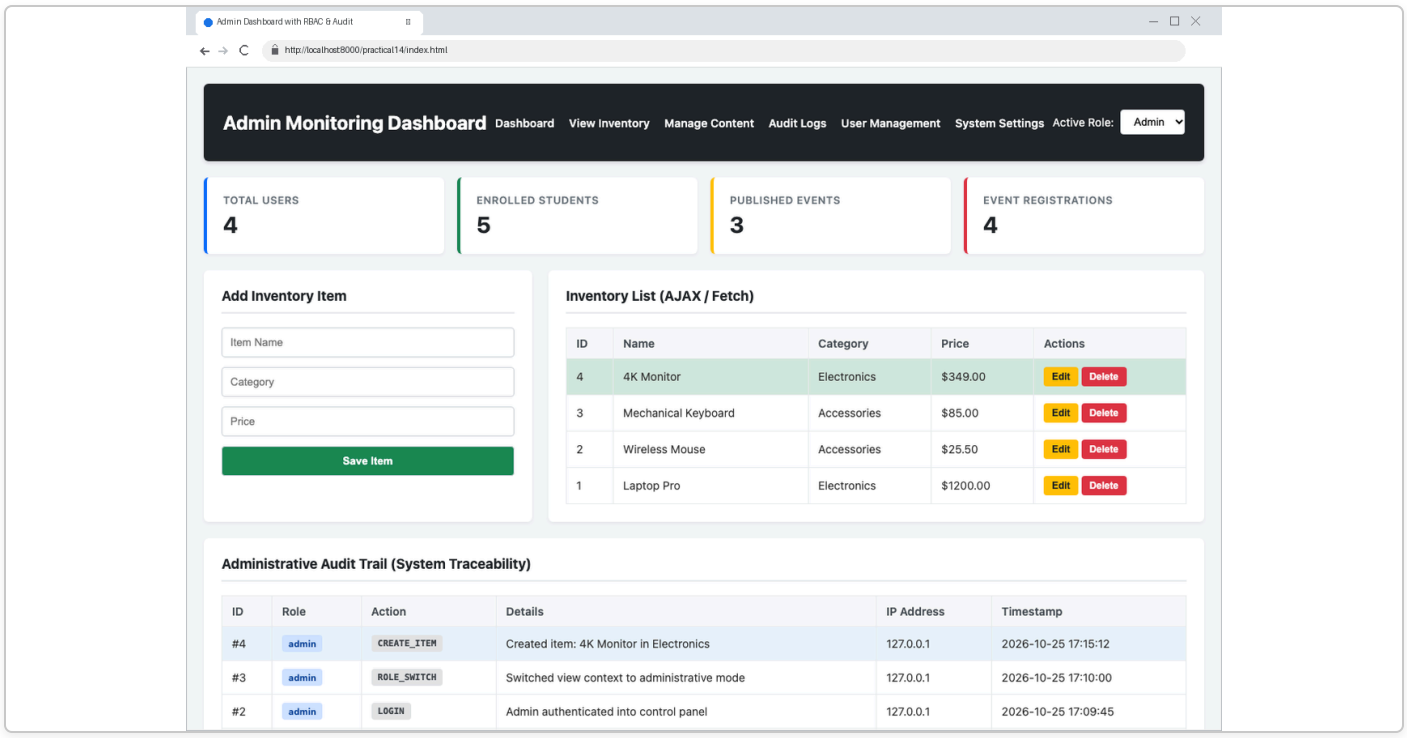
A. Test Case 1 – Admin Dashboard Overview with Dynamic Aggregate Stat Cards (Users, Students, Events, Registrations)



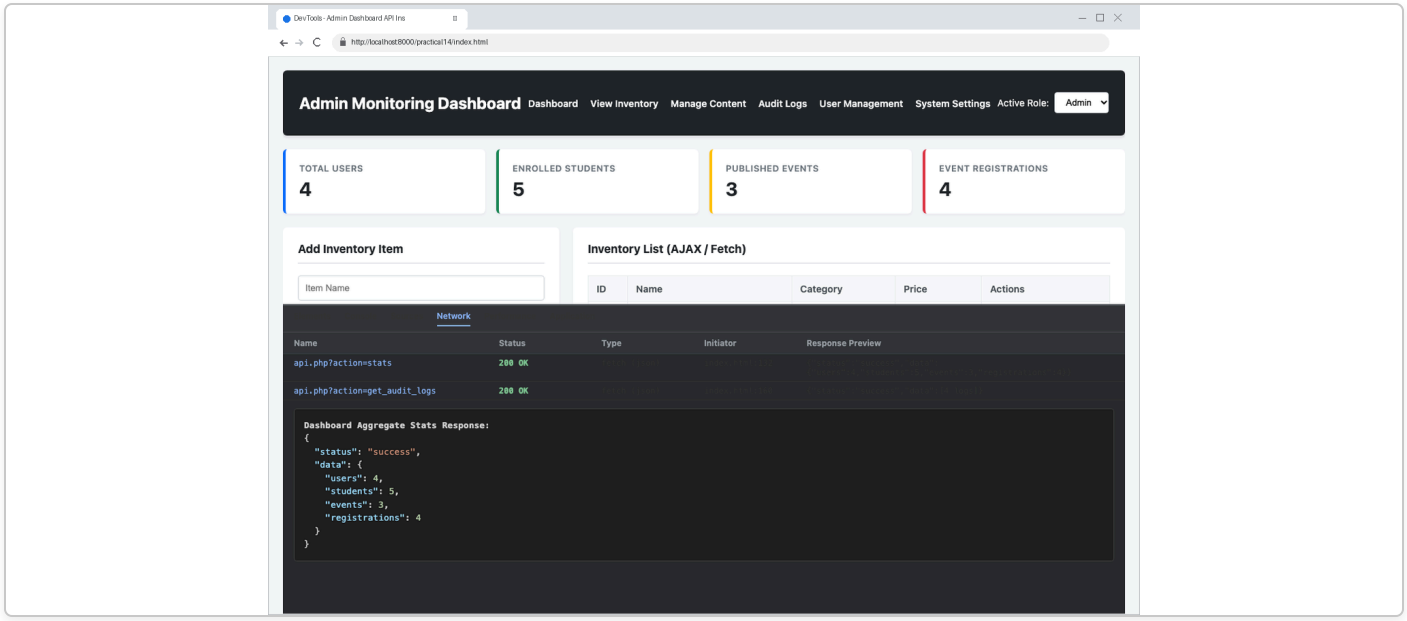
B. Test Case 2 – Dynamic RBAC Navigation Menu (Editor Role Perspective with 3 Links)



C. Test Case 3 – Live Administrative Audit Log Trail & Real-Time Event Recording



D. Test Case 4 – DevTools Network Tab & RESTful JSON Stats API Response Inspection



Github link:
https://github.com/Bhavy901/student-hub

Learning Outcome

- Understood how to execute aggregated SQL queries (COUNT(*)) across multiple tables to feed administrative summary dashboards.
- Designed an immutable system audit trail to record every administrative mutation, capturing timestamps, actor roles, detailed event messages, and client IP addresses.
- Configured dynamic navigation systems where accessible application modules depend strictly on verified authorization levels.
- Connected frontend asynchronous UI events with backend audit logging pipelines for unified application state